

PROJET HAUTE DISPONIBILITÉ

HAProxy + Heartbeat

Haute disponibilité et répartition de charge

Infrastructure web hautement disponible : HAProxy répartit les requêtes entre deux serveurs web, Heartbeat fait basculer une IP virtuelle en cas de panne.

HAProxy

Heartbeat

Apache2 / PHP

VMware Workstation

Younes FRI & Saidou DIA

BTS SIO SISR

Sommaire

Introduction	3
Architecture de l'infrastructure	4
2.1 Présentation générale	4
2.2 Tableau des machines virtuelles	4
2.3 Principe de fonctionnement	5
Préparation de l'environnement VMware	6
3.1 Création des segments réseau WAN et LAN	6
3.2 Configuration des adresses IP statiques	8
3.3 Vérification de la connectivité réseau	11
Déploiement des serveurs web (SERV-WEB1 et SERV-WEB2)	18
4.1 Installation d'Apache2 et PHP	18
4.2 Page d'accueil distincte sur chaque serveur	18
4.3 Vérification avec curl depuis les load balancers	19
Configuration de HAProxy (Load Balancing)	20
5.1 Présentation de HAProxy	20
5.2 Fichier de configuration haproxy.cfg	20
Configuration de Heartbeat (Haute Disponibilité)	23
6.1 Présentation de Heartbeat	23
6.2 Fichier ha.cf — Configuration du cluster	23
6.3 Fichier haresources — Définition des ressources	24
6.4 Fichier authkeys — Authentification entre nœuds	25
Tests de bascule et validation	26
7.1 Test 1 — Round-robin entre SERV-WEB1 et SERV-WEB2	26
7.2 Page de statistiques HAProxy	28
7.3 Test 2 — Panne de HAProxy1 : bascule de la VIP	28
7.4 Vérification du statut Heartbeat sur le nœud actif	30
7.5 Test 3 — Panne d'un serveur backend	30
7.6 Test 4 — Retour à la normale (auto_failback)	31
Bilan et retour d'expérience	32
8.1 Ce qui a été mis en place	32
8.2 Limites de l'architecture	32

Introduction

Dans le cadre du BTS SIO option SISR, ce projet a pour objectif la mise en place d'une infrastructure web hautement disponible combinant deux outils complémentaires :

- **HAProxy** : outil de load balancing (répartition de charge) qui distribue les requêtes HTTP entre plusieurs serveurs web backend.
- **Heartbeat** : outil de haute disponibilité qui gère une adresse IP Virtuelle (VIP) flottante entre deux nœuds, garantissant la continuité du service en cas de panne.

L'ensemble de l'infrastructure est déployé sur VMware Workstation avec cinq machines virtuelles réparties sur deux réseaux distincts (WAN et LAN).

L'objectif final est qu'un client (CLI-01) puisse toujours accéder au service web via la VIP 192.168.0.10, même si l'un des serveurs tombe en panne.

Architecture de l'infrastructure

2.1 Présentation générale

L'infrastructure repose sur deux réseaux VMware isolés :

- Réseau WAN (192.168.0.0/24) : relie le client CLI-01 aux deux load balancers HAProxy1 et HAProxy2.
- Réseau LAN (172.20.0.0/24) : réseau interne reliant les load balancers aux serveurs web SERV-WEB1 et SERV-WEB2.

La VIP 192.168.0.10 est une adresse IP flottante gérée par Heartbeat. Elle est portée par HAProxy1 (nœud primaire) et bascule automatiquement vers HAProxy2 (nœud secondaire) en cas de défaillance.

2.2 Tableau des machines virtuelles

VM	Rôle	IP WAN	IP LAN	RAM / vCPU
CLI-01	Client de test	192.168.0.100/24	—	1 Go / 1
HAProxy1	Load balancer primaire	192.168.0.11/24	172.20.0.11/24	1 Go / 1
HAProxy2	Load balancer secondaire	192.168.0.12/24	172.20.0.12/24	1 Go / 1
SERV-WEB1	Serveur web Apache2/PHP	—	172.20.0.21/24	1 Go / 1
SERV-WEB2	Serveur web Apache2/PHP	—	172.20.0.22/24	1 Go / 1

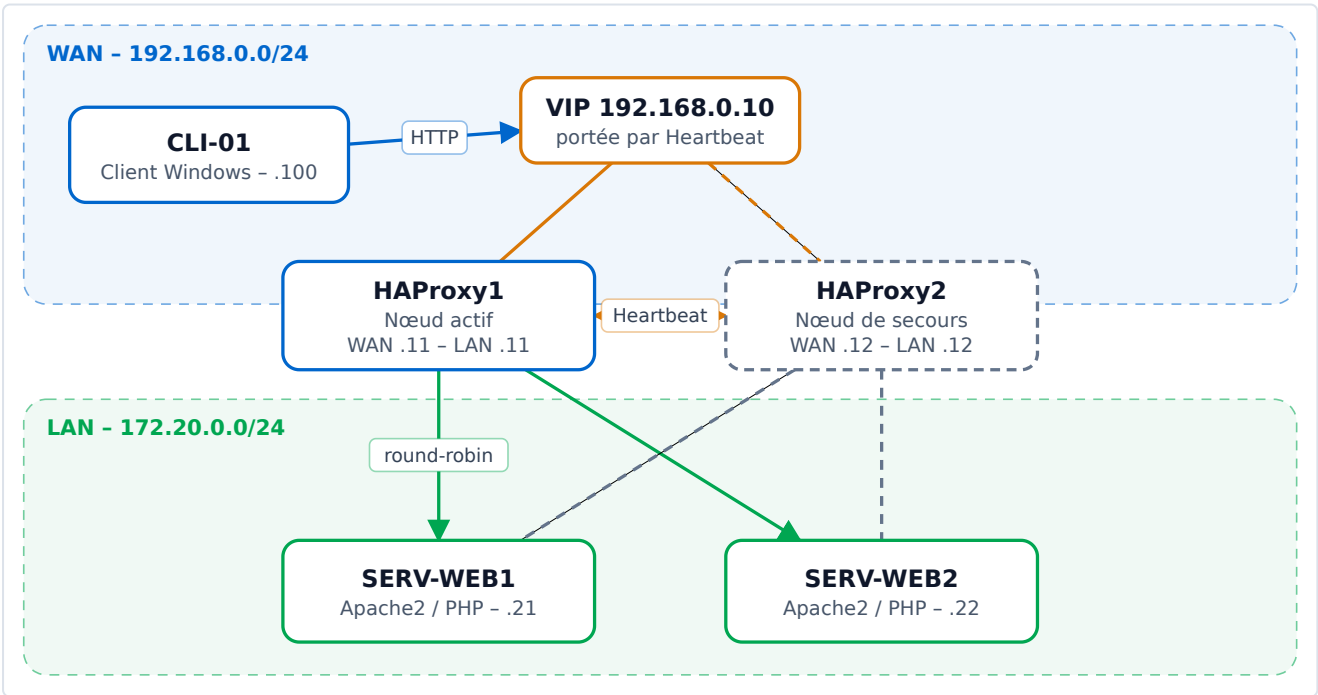


Figure 1 — Schéma de l'infrastructure : VIP, load balancers HAProxy et serveurs web

2.3 Principe de fonctionnement

Flux d'une requête client

1. CLI-01 envoie une requête HTTP vers la VIP 192.168.0.10
2. Heartbeat garantit que la VIP est portée par HAProxy1 (ou HAProxy2 en cas de panne)
3. HAProxy reçoit la requête et la distribue en round-robin vers SERV-WEB1 ou SERV-WEB2
4. Le serveur web backend traite la requête et renvoie la réponse
5. HAProxy transmet la réponse au client CLI-01

Préparation de l'environnement VMware

3.1 Création des segments réseau WAN et LAN

Avant de créer les machines virtuelles, il faut configurer deux segments réseau virtuels dans VMware Workstation : un segment WAN (192.168.0.0/24) et un segment LAN (172.20.0.0/24).

Dans les paramètres d'une VM, on utilise l'assistant d'ajout de matériel pour ajouter une seconde carte réseau. HAProxy1 et HAProxy2 ont chacun deux interfaces réseau : une sur le WAN et une sur le LAN.

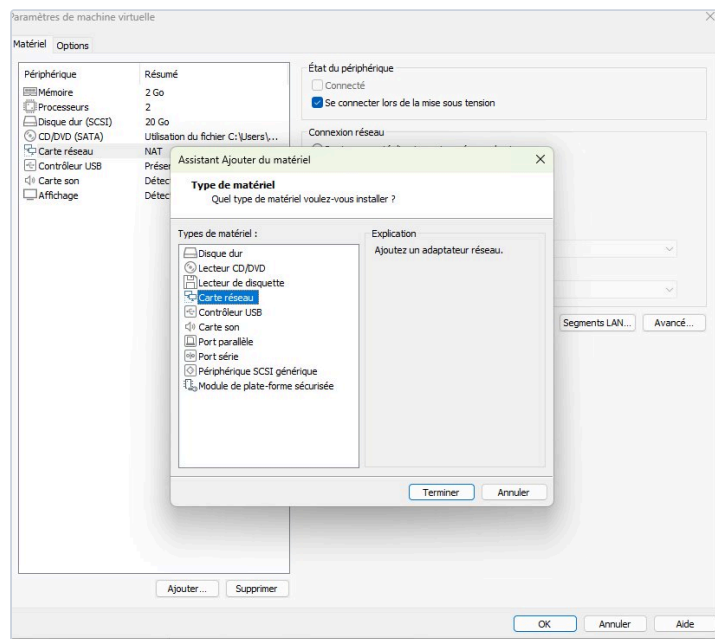


Figure 2 — Ajout d'une seconde carte réseau sur HAProxy1

Ce que je fais : J'ouvre les paramètres de la VM HAProxy1 (puis HAProxy2) et j'ajoute une seconde carte réseau via l'Assistant Ajouter du matériel → Carte réseau.

Pourquoi : HAProxy doit recevoir les requêtes depuis le réseau public (WAN, ens33) et les transmettre aux serveurs web situés sur le réseau privé (LAN, ens37). Deux interfaces sont donc nécessaires sur chaque load balancer.

Les segments LAN globaux sont définis dans VMware Workstation. On crée ici deux segments nommés WAN et LAN, qui seront assignés aux interfaces des VMs.

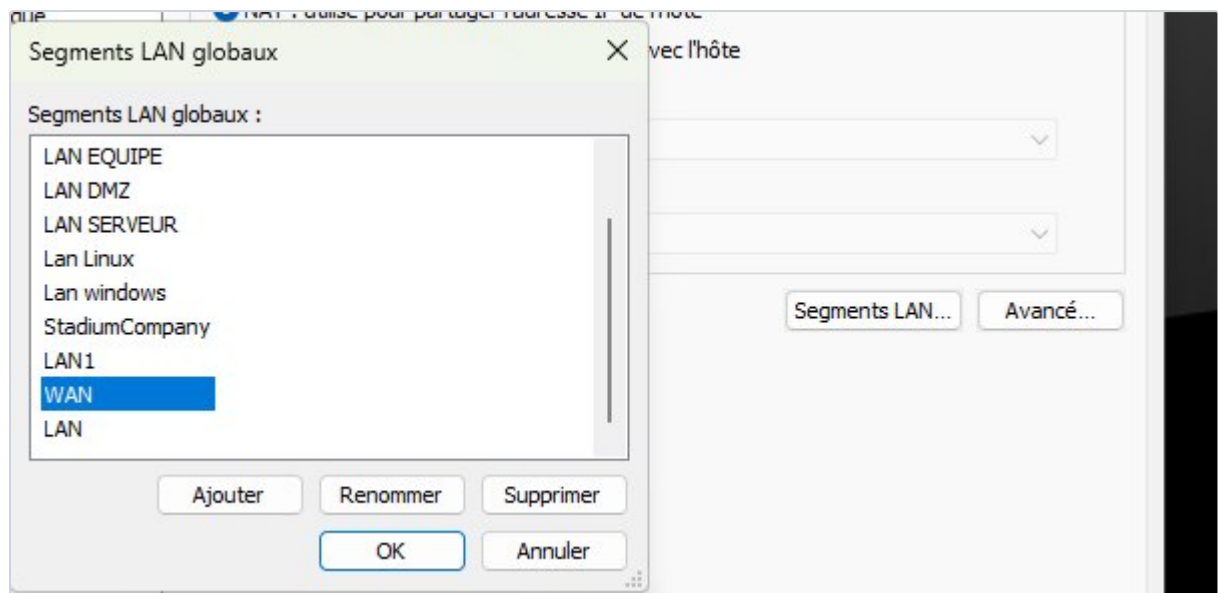


Figure 3 — Création des segments LAN globaux WAN et LAN dans VMware Workstation

Ce que je fais : Je crée deux segments LAN globaux : WAN (pour le réseau public 192.168.0.0/24) et LAN (pour le réseau privé 172.20.0.0/24) via Édition → Éditeur de réseau virtuel → Segments LAN...

Pourquoi : Les segments globaux permettent d'isoler totalement les VMs du reste de la machine hôte. Cela reproduit une vraie topologie d'entreprise avec une DMZ (WAN) et un réseau interne (LAN) non routables vers l'extérieur.

3.2 Configuration des adresses IP statiques

Chaque VM doit être configurée avec une adresse IP statique dans le fichier `/etc/network/interfaces` (sous Debian/Ubuntu). Voici les configurations de chaque machine.

HAProxy1 — 192.168.0.11 (WAN) / 172.20.0.11 (LAN)

HAProxy1 est le nœud primaire. Il dispose de deux interfaces : `ens33` sur le WAN et `ens37` sur le LAN.

```
GNU nano 8.4 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens33
iface ens33 inet static
address 192.168.0.11/24

#ens37
allow-hotplug ens37
iface ens37 inet static
address 172.20.0.11/24
```

Figure 4 — Configuration réseau de HAProxy1 dans `/etc/network/interfaces`

Ce que je fais : J'édite `/etc/network/interfaces` avec nano et je déclare les deux interfaces `ens33` (192.168.0.11/24) et `ens37` (172.20.0.11/24) en adressage statique, puis je redémarre le réseau avec `systemctl restart networking`.

Pourquoi : HAProxy doit avoir des adresses IP fixes et prévisibles. En DHCP, les IP pourraient changer au redémarrage et Heartbeat comme HAProxy ne fonctionneraient plus. Le statique garantit la stabilité du cluster.

Explication des directives :

- **allow-hotplug ens33** : active l'interface ens33 automatiquement au démarrage.
- **iface ens33 inet static** : configure l'interface en IP statique (pas de DHCP).
- **address 192.168.0.11/24** : adresse IP fixe sur le réseau WAN avec un masque /24 (255.255.255.0).
- De même pour ens37 avec l'adresse 172.20.0.11/24 sur le réseau LAN.

HAProxy2 — 192.168.0.12 (WAN) / 172.20.0.12 (LAN)

HAProxy2 est le nœud secondaire. Sa configuration est identique à HAProxy1 avec des adresses IP différentes.

```
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens33
iface ens33 inet static
address 192.168.0.12/24

#ens37
allow-hotplug ens37
iface ens37 inet static
address 172.20.0.12/24
```

Figure 5 — Configuration réseau de HAProxy2

Ce que je fais : Je configure HAProxy2 exactement comme HAProxy1 mais avec les IP .12 sur le WAN et le LAN.

Pourquoi : Les deux nœuds du cluster doivent être symétriques : mêmes interfaces, même plage réseau, même rôle potentiel. Seules les adresses IP propres diffèrent. C'est indispensable pour que Heartbeat puisse faire basculer la VIP entre les deux.

SERV-WEB1 — 172.20.0.21 (LAN uniquement)

Les serveurs web n'ont qu'une seule interface réseau sur le LAN. Ils ne sont pas directement accessibles depuis l'extérieur (le WAN), uniquement via HAProxy.

```
GNU nano 2.9.3 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens33
iface ens33 inet static
address 172.20.0.21/24
```

Figure 6 — Configuration réseau de SERV-WEB1

Ce que je fais : J'attribue à SERV-WEB1 une seule interface ens33 en 172.20.0.21/24 sur le LAN.

Pourquoi : Les serveurs backend ne doivent jamais être joignables directement depuis Internet/WAN pour des raisons de sécurité. Ils ne doivent répondre qu'aux requêtes relayées par HAProxy. C'est le principe d'une architecture n-tiers.

SERV-WEB2 — 172.20.0.22 (LAN uniquement)

```
GNU nano 8.4 /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug ens33
iface ens33 inet static
address 172.20.0.22/24
```

Figure 7 — Configuration réseau de SERV-WEB2

Ce que je fais : Même configuration que SERV-WEB1 avec l'adresse 172.20.0.22/24.

Pourquoi : Avoir deux serveurs backend identiques permet à HAProxy de répartir la charge (load balancing). Si l'un tombe, l'autre prend tout le trafic grâce aux health-checks.

3.3 Vérification de la connectivité réseau

Après configuration, on vérifie que les interfaces sont bien actives avec la commande `ip a` sur chaque nœud HAProxy.

Vérification sur HAProxy1

On observe les interfaces `ens33` (192.168.0.11) et `ens37` (172.20.0.11). La VIP 192.168.0.10 est également visible car Heartbeat est déjà démarré — elle est portée par le nœud primaire.

```
root@HAProxy1:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:96:f4:47 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altnamename enx00c2996f447
    inet 192.168.0.11/24 brd 192.168.0.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.181.147/24 brd 192.168.181.255 scope global dynamic noprefixroute ens33
        valid_lft 1435sec preferred_lft 1210sec
    inet 192.168.0.10/24 scope global secondary ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::d637:6d0a:1296:f413/64 scope link
        valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:96:f4:51 brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altnamename enx00c2996f451
    inet 172.20.0.11/24 brd 172.20.0.255 scope global ens37
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:391f:1e96:1431/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@HAProxy1:~#
```

Figure 8 — Interfaces de HAProxy1 : la VIP 192.168.0.10 est présente sur ens33

Ce que je fais : J'exécute `ip a` sur HAProxy1 et je vérifie que `ens33` porte bien deux adresses : 192.168.0.11/24 (IP fixe du nœud) et 192.168.0.10/24 (scope global secondary) qui est la VIP gérée par Heartbeat.

Pourquoi : La présence de la VIP en tant qu'alias secondaire sur `ens33` prouve que Heartbeat fonctionne et que HAProxy1 est bien le nœud actif. C'est la signature réseau du cluster au repos.

Vérification sur HAProxy2

HAProxy2 affiche ses interfaces ens33 (192.168.0.12) et ens37 (172.20.0.12). La VIP n'est pas présente ici car HAProxy1 est actif.

```
root@HAProxy2:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c9:83:e0 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c29c983e0
    inet 192.168.0.12/24 brd 192.168.0.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe9:83e0/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c9:83:ea brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altname enx00c29c983ea
    inet 172.20.0.12/24 brd 172.20.0.255 scope global ens37
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe9:83ea/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@HAProxy2:~#
```

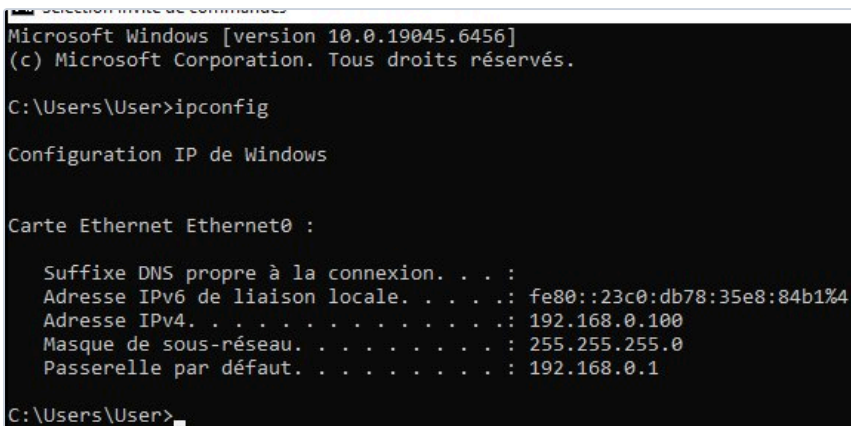
Figure 9 — Interfaces de HAProxy2 : pas de VIP, le nœud est passif

Ce que je fais : Je fais ip a sur HAProxy2 et je constate qu'il n'y a QUE les deux IP de base (192.168.0.12 et 172.20.0.12). La VIP n'apparaît pas.

Pourquoi : C'est la preuve que le cluster est en mode actif/passif : un seul nœud détient la VIP à la fois. Si les deux l'avaient simultanément, on aurait un conflit ARP sur le réseau (situation dite de split-brain).

Vérification sur CLI-01 (Windows)

Le client CLI-01 sous Windows dispose de l'adresse 192.168.0.100/24 sur le réseau WAN, ce qui lui permet d'atteindre la VIP 192.168.0.10.



```
Microsoft Windows [version 10.0.19045.6456]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\User>ipconfig

Configuration IP de Windows

Carte Ethernet Ethernet0 :

    Suffixe DNS propre à la connexion. . . . :
    Adresse IPv6 de liaison locale. . . . . : fe80::23c0:db78:35e8:84b1%4
    Adresse IPv4. . . . . : 192.168.0.100
    Masque de sous-réseau. . . . . : 255.255.255.0
    Passerelle par défaut. . . . . : 192.168.0.1

C:\Users\User>
```

Figure 10 — Configuration IP du client CLI-01 (192.168.0.100)

Ce que je fais : J'ouvre une invite de commandes sur le client Windows et je lance ipconfig pour vérifier que son IPv4 est bien 192.168.0.100 avec un masque /24.

Pourquoi : Le client doit se trouver dans le même sous-réseau que la VIP (192.168.0.0/24) pour pouvoir y accéder en couche 2 (ARP) sans passer par un routeur. C'est la topologie la plus simple pour valider une HA.

Ping depuis HAProxy1 vers les serveurs web

Depuis HAProxy1, on vérifie la connectivité vers SERV-WEB1 (172.20.0.21) et SERV-WEB2 (172.20.0.22). Les deux pings aboutissent, confirmant que le réseau LAN est fonctionnel et que HAProxy1 peut atteindre ses backends.

```
root@HAProxy1:~# ping 172.20.0.21
PING 172.20.0.21 (172.20.0.21) 56(84) bytes of data.
64 bytes from 172.20.0.21: icmp_seq=1 ttl=64 time=0.449 ms
64 bytes from 172.20.0.21: icmp_seq=2 ttl=64 time=0.773 ms
64 bytes from 172.20.0.21: icmp_seq=3 ttl=64 time=0.610 ms
64 bytes from 172.20.0.21: icmp_seq=4 ttl=64 time=0.764 ms
64 bytes from 172.20.0.21: icmp_seq=5 ttl=64 time=0.886 ms
64 bytes from 172.20.0.21: icmp_seq=6 ttl=64 time=0.722 ms
^Z
[2]+  Stoppé                  ping 172.20.0.21
root@HAProxy1:~# ping 172.20.0.22
PING 172.20.0.22 (172.20.0.22) 56(84) bytes of data.
64 bytes from 172.20.0.22: icmp_seq=1 ttl=64 time=0.840 ms
64 bytes from 172.20.0.22: icmp_seq=2 ttl=64 time=0.586 ms
64 bytes from 172.20.0.22: icmp_seq=3 ttl=64 time=0.615 ms
64 bytes from 172.20.0.22: icmp_seq=4 ttl=64 time=0.709 ms
^Z
[3]+  Stoppé                  ping 172.20.0.22
root@HAProxy1:~#
```

Figure 11 — Ping de HAProxy1 vers SERV-WEB1 et SERV-WEB2

Ce que je fais : Je lance ping 172.20.0.21 puis ping 172.20.0.22 depuis HAProxy1 pour tester la connectivité LAN avec chacun des backends.

Pourquoi : Avant de configurer le load balancing, il faut absolument valider que HAProxy peut joindre les serveurs web. Sans cela, les requêtes tomberaient en timeout et les health-checks marqueraient les backends comme DOWN.

Ping depuis HAProxy2 vers les serveurs web

Même vérification depuis HAProxy2 vers SERV-WEB2 (172.20.0.22) puis SERV-WEB1 (172.20.0.21). La connectivité est établie, ce qui garantit que le nœud secondaire peut aussi router les requêtes vers les backends en cas de bascule.

```
root@HAProxy2:~# ping 172.20.0.22
PING 172.20.0.22 (172.20.0.22) 56(84) bytes of data.
64 bytes from 172.20.0.22: icmp_seq=1 ttl=64 time=0.827 ms
64 bytes from 172.20.0.22: icmp_seq=2 ttl=64 time=0.733 ms
64 bytes from 172.20.0.22: icmp_seq=3 ttl=64 time=0.745 ms
64 bytes from 172.20.0.22: icmp_seq=4 ttl=64 time=0.727 ms
^C
[1]+  Stoppé                  ping 172.20.0.22
root@HAProxy2:~# ping 172.20.0.21
PING 172.20.0.21 (172.20.0.21) 56(84) bytes of data.
64 bytes from 172.20.0.21: icmp_seq=1 ttl=64 time=0.432 ms
64 bytes from 172.20.0.21: icmp_seq=2 ttl=64 time=0.509 ms
64 bytes from 172.20.0.21: icmp_seq=3 ttl=64 time=0.758 ms
64 bytes from 172.20.0.21: icmp_seq=4 ttl=64 time=0.763 ms
^C
[2]+  Stoppé                  ping 172.20.0.21
root@HAProxy2:~#
```

Figure 12 — Ping de HAProxy2 vers les deux serveurs web

Ce que je fais : Je refais la même série de ping mais depuis HAProxy2 cette fois.

Pourquoi : Si HAProxy2 ne pouvait pas joindre les backends, la bascule Heartbeat serait inutile : la VIP se déplacerait mais le service tomberait quand même. Ce test prouve que HAProxy2 est un vrai nœud de secours opérationnel.

Ping depuis SERV-WEB2 vers les load balancers

Depuis SERV-WEB2, on ping HAProxy2 (172.20.0.12) et SERV-WEB2 lui-même (172.20.0.22). Cela vérifie que les serveurs web sont bien connectés au réseau LAN et peuvent communiquer avec les load balancers.

```
root@SRV-WEB2:~# ping 172.20.0.12
PING 172.20.0.12 (172.20.0.12) 56(84) bytes of data.
64 bytes from 172.20.0.12: icmp_seq=1 ttl=64 time=1.58 ms
64 bytes from 172.20.0.12: icmp_seq=2 ttl=64 time=0.867 ms
64 bytes from 172.20.0.12: icmp_seq=3 ttl=64 time=0.551 ms
64 bytes from 172.20.0.12: icmp_seq=4 ttl=64 time=0.736 ms
^Z
[1]+  Stoppé                  ping 172.20.0.12
root@SRV-WEB2:~# ping 172.20.0.22
PING 172.20.0.22 (172.20.0.22) 56(84) bytes of data.
64 bytes from 172.20.0.22: icmp_seq=1 ttl=64 time=0.096 ms
64 bytes from 172.20.0.22: icmp_seq=2 ttl=64 time=0.035 ms
64 bytes from 172.20.0.22: icmp_seq=3 ttl=64 time=0.033 ms
64 bytes from 172.20.0.22: icmp_seq=4 ttl=64 time=0.048 ms
^Z
[2]+  Stoppé                  ping 172.20.0.22
root@SRV-WEB2:~# _
```

Figure 13 — Ping depuis SERV-WEB2

Ce que je fais : Je me connecte sur SERV-WEB2 et je fais les ping en sens inverse pour confirmer que la communication est bien bidirectionnelle.

Pourquoi : Une route doit exister dans les deux sens pour qu'une session TCP s'établisse. Un ping qui ne passe que dans un sens trahit un problème de routage ou de pare-feu qui empêcherait HAProxy de recevoir les réponses HTTP.

Ping depuis SERV-WEB1 vers les load balancers

Depuis SERV-WEB1, on ping HAProxy1 (172.20.0.11) et SERV-WEB1 lui-même (172.20.0.21). Tous les nœuds du réseau LAN communiquent correctement entre eux.

```
root@SRV-WEB1:~# ping 172.20.0.11
PING 172.20.0.11 (172.20.0.11) 56(84) bytes of data.
64 bytes from 172.20.0.11: icmp_seq=1 ttl=64 time=0.412 ms
64 bytes from 172.20.0.11: icmp_seq=2 ttl=64 time=0.463 ms
64 bytes from 172.20.0.11: icmp_seq=3 ttl=64 time=0.567 ms
64 bytes from 172.20.0.11: icmp_seq=4 ttl=64 time=0.526 ms
^Z
[2]+  Stoppé                  ping 172.20.0.11
root@SRV-WEB1:~# ping 172.20.0.21
PING 172.20.0.21 (172.20.0.21) 56(84) bytes of data.
64 bytes from 172.20.0.21: icmp_seq=1 ttl=64 time=0.021 ms
64 bytes from 172.20.0.21: icmp_seq=2 ttl=64 time=0.045 ms
64 bytes from 172.20.0.21: icmp_seq=3 ttl=64 time=0.110 ms
^Z
[3]+  Stoppé                  ping 172.20.0.21
root@SRV-WEB1:~#
```

Figure 14 — Ping depuis SERV-WEB1

Ce que je fais : Je termine la matrice de ping depuis SERV-WEB1.

Pourquoi : Cela clôt la phase de validation réseau : tous les couples (HAProxy1/2 ↔ SERV-WEB1/2) sont joignables dans les deux sens. Le LAN est donc 100 % opérationnel avant d'attaquer la configuration applicative.

Déploiement des serveurs web (SERV-WEB1 et SERV-WEB2)

Les deux serveurs backend hébergent le service web à rendre disponible. Ils sont installés avec Apache2 et PHP, et chacun affiche une page d'accueil distincte pour permettre d'identifier visuellement quel serveur répond à une requête.

4.1 Installation d'Apache2 et PHP

Sur SERV-WEB1 et SERV-WEB2, on installe le serveur web Apache2 et le module PHP avec les commandes suivantes :

```
sudo apt update
sudo apt install apache2 php libapache2-mod-php -y
sudo systemctl enable --now apache2
```

Ce que je fais : J'installe les paquets apache2, php et libapache2-mod-php puis j'active le service Apache pour qu'il démarre automatiquement.

Pourquoi : Apache2 est le serveur HTTP qui va traiter les requêtes relayées par HAProxy. PHP permet de générer une page dynamique affichant le nom du serveur, ce qui sera indispensable pour visualiser le round-robin.

4.2 Page d'accueil distincte sur chaque serveur

Pour prouver que le load balancing fonctionne, chaque serveur doit renvoyer une page identifiable. On remplace `/var/www/html/index.html` par une page contenant le hostname et une mise en forme Bootstrap (via CDN).

Sur SERV-WEB1 on crée un fichier `index.html` qui affiche "OK WEB 1" et sur SERV-WEB2 un fichier qui affiche "OK WEB 2".

```
sudo nano /var/www/html/index.html
```

Ce que je fais : Sur chaque serveur, je crée une page HTML différente avec un titre Bootstrap bien visible : "OK WEB 1" pour SERV-WEB1 et "OK WEB 2" pour SERV-WEB2.

Pourquoi : Sans différence visible entre les deux pages, on ne pourrait pas démontrer que les requêtes sont bien réparties. C'est une technique standard en TP de HA pour valider le round-robin de façon visuelle.

4.3 Vérification avec curl depuis les load balancers

Avant de configurer HAProxy, on valide que les pages sont bien accessibles depuis HAProxy1 et HAProxy2 :

```
curl http://172.20.0.21  
curl http://172.20.0.22
```

Ce que je fais : Je fais un curl depuis chaque HAProxy vers chaque SERV-WEB pour vérifier qu'Apache répond correctement en HTTP.

Pourquoi : Un ping qui passe ne suffit pas : il faut aussi que le port TCP/80 soit ouvert et qu'Apache serve bien la page. Si curl ne renvoie rien, HAProxy basculera en erreur dès qu'il fera son health-check.

Configuration de HAProxy (Load Balancing)

5.1 Présentation de HAProxy

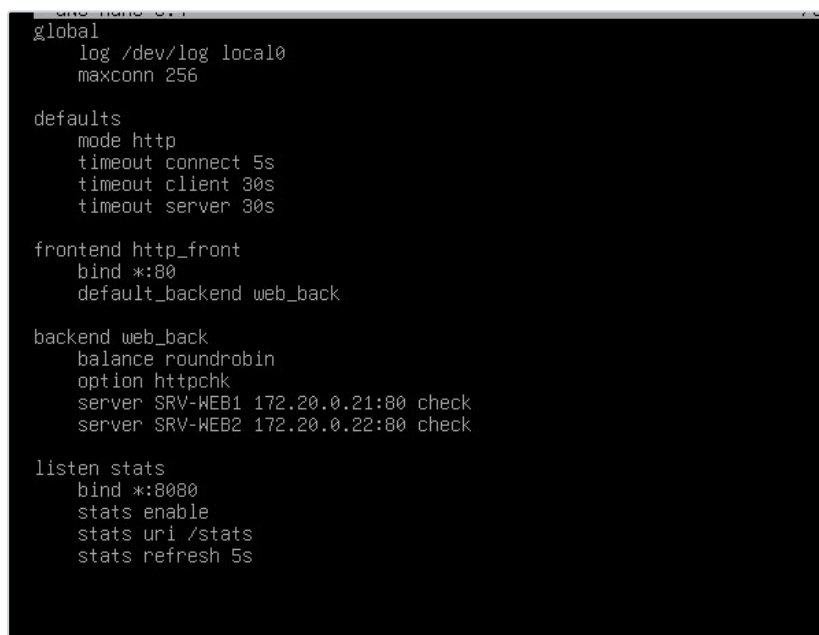
HAProxy (High Availability Proxy) est un logiciel open-source de load balancing et de proxy TCP/HTTP. Il reçoit les connexions entrantes et les distribue entre plusieurs serveurs backend selon un algorithme défini (ici : round-robin).

HAProxy est installé et configuré de façon identique sur HAProxy1 et HAProxy2. C'est Heartbeat qui décide lequel des deux est actif en portant la VIP.

```
sudo apt install haproxy -y
```

5.2 Fichier de configuration haproxy.cfg

Le fichier de configuration principal se trouve dans `/etc/haproxy/haproxy.cfg`. Il est composé de plusieurs blocs.



```
global
    log /dev/log local0
    maxconn 256

defaults
    mode http
    timeout connect 5s
    timeout client 30s
    timeout server 30s

frontend http_front
    bind *:80
    default_backend web_back

backend web_back
    balance roundrobin
    option httpchk
    server SRV-WEB1 172.20.0.21:80 check
    server SRV-WEB2 172.20.0.22:80 check

listen stats
    bind *:8080
    stats enable
    stats uri /stats
    stats refresh 5s
```

Figure 15 — Fichier `/etc/haproxy/haproxy.cfg`

Ce que je fais : J'édite `/etc/haproxy/haproxy.cfg` pour définir les blocs `global`, `defaults`, `frontend`, `backend` et `listen stats`. Le fichier est strictement identique sur HAProxy1 et HAProxy2.

Pourquoi : Les deux nœuds doivent avoir la même configuration car n'importe lequel peut devenir actif à tout moment. Si les configurations divergeaient, le comportement du service changerait au moment d'une bascule, ce qui casserait la transparence pour le client.

Bloc global

```
global
log /dev/log local0
maxconn 256
```

- **log /dev/log local0** : redirige les logs vers le journal système (syslog).
- **maxconn 256** : limite le nombre maximum de connexions simultanées à 256.

Bloc defaults

```
defaults
mode http
timeout connect 5s
timeout client 30s
timeout server 30s
```

- **mode http** : HAProxy fonctionne en mode HTTP (niveau 7, applicatif).
- **timeout connect 5s** : délai maximum pour établir une connexion vers un backend.
- **timeout client 30s** : délai d'inactivité côté client avant fermeture.
- **timeout server 30s** : délai d'inactivité côté serveur avant fermeture.

Bloc frontend

```
frontend http_front
bind *:80
default_backend web_back
```

- **bind *:80** : HAProxy écoute sur toutes les interfaces réseau sur le port 80 (HTTP standard).
- **default_backend web_back** : redirige toutes les requêtes vers le groupe de serveurs nommé web_back.

Bloc backend

```
backend web_back
balance roundrobin
option httpchk GET /
server SRV-WEB1 172.20.0.21:80 check
server SRV-WEB2 172.20.0.22:80 check
```

- **balance roundrobin** : les requêtes sont distribuées à tour de rôle entre les serveurs (1 requête sur SRV-WEB1, suivante sur SRV-WEB2, etc.).
- **option httpchk GET /** : HAProxy vérifie régulièrement que chaque serveur répond à une requête HTTP GET sur /. Si un serveur ne répond plus, il est retiré automatiquement du pool.
- **server SRV-WEB1 172.20.0.21:80 check** : déclare le premier serveur backend avec son IP LAN et active les health checks.
- **server SRV-WEB2 172.20.0.22:80 check** : même chose pour le second serveur.

Bloc listen stats (page de statistiques)

```
listen stats
bind *:8080
stats enable
stats uri /stats
stats refresh 5s
```

- **bind *:8080** : la page de stats est accessible sur le port 8080.
- **stats uri /stats** : accès via l'URL <http://192.168.0.10:8080/stats>.
- **stats refresh 5s** : la page se rafraîchit automatiquement toutes les 5 secondes.

Configuration de Heartbeat (Haute Disponibilité)

6.1 Présentation de Heartbeat

Heartbeat est un logiciel qui permet la mise en place d'un cluster actif/passif. Il surveille l'état des nœuds en s'envoyant des messages réguliers (heartbeats). Si le nœud primaire ne répond plus, le nœud secondaire prend automatiquement en charge la VIP et les services associés.

Heartbeat nécessite trois fichiers de configuration, identiques sur les deux nœuds :

- `/etc/ha.d/ha.cf` : fichier principal de configuration du cluster.
- `/etc/ha.d/haresources` : définit le nœud primaire et les ressources à gérer.
- `/etc/ha.d/authkeys` : clé d'authentification partagée entre les nœuds.

```
sudo apt install heartbeat -y
```

6.2 Fichier ha.cf — Configuration du cluster

Ce fichier définit les paramètres de communication entre les deux nœuds Heartbeat.



```
GNU nano 8.4 /etc/ha.d/ha.cf
logfile /var/log/heartbeat.log
logfacility local0
keepalive 2
deadtime 10
bcast ens33
node HAProxy1
node HAProxy2
auto_failback on
```

Figure 16 — Fichier `/etc/ha.d/ha.cf`

Ce que je fais : Je crée `/etc/ha.d/ha.cf` avec les directives `keepalive`, `deadtime`, `bcast`, `node` et `auto_failback`. Le même fichier est copié sur `HAProxy1` et `HAProxy2`.

Pourquoi : C'est le cœur de la configuration : il dit comment les nœuds communiquent (`bcast ens33`), à quelle fréquence (`keepalive 2`), au bout de combien de temps de silence on déclare un nœud mort (`deadtime 10`) et qui est primaire. Sans cela, aucune bascule ne serait possible.

Explication de chaque directive :

```
logfile /var/log/heartbeat.log
```

- **logfile** : chemin du fichier de log Heartbeat. Permet de suivre les événements de bascule.

```
logfacility local0
```

- **logfacility** : envoie également les logs vers syslog avec la facility local0.

```
keepalive 2
```

- **keepalive 2** : Heartbeat envoie un message de vie (heartbeat) toutes les 2 secondes.

```
deadtime 10
```

- **deadtime 10** : si un nœud ne répond pas pendant 10 secondes, il est déclaré mort et la bascule se déclenche.

```
bcast ens33
```

- **bcast ens33** : les messages heartbeat sont diffusés en broadcast sur l'interface ens33 (réseau WAN).

```
node HAProxy1  
node HAProxy2
```

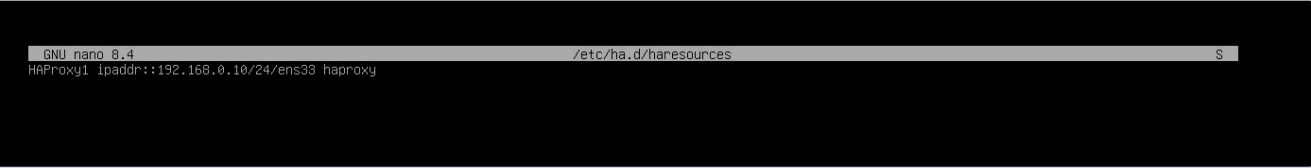
- **node** : déclare les deux nœuds du cluster. Les noms doivent correspondre exactement au hostname des machines (commande hostname).

```
auto_failback on
```

- **auto_failback on** : quand le nœud primaire (HAProxy1) revient en ligne, il reprend automatiquement la VIP. Si désactivé (off), la VIP resterait sur HAProxy2 même après la reprise.

6.3 Fichier haresources — Définition des ressources

Ce fichier déclare quel nœud est le nœud primaire et quelles ressources il doit gérer (VIP + service HAProxy).



```
GNU nano 8.4 /etc/ha.d/haresources  
HAProxy1 ipaddr::192.168.0.10/24/ens33 haproxy
```

Figure 17 — Fichier /etc/ha.d/haresources

Ce que je fais : J'écris dans `/etc/ha.d/haresources` la ligne : `HAProxy1 ipaddr::192.168.0.10/24/ens33 haproxy`, identique sur les deux nœuds.

Pourquoi : Cette ligne dit à Heartbeat trois choses : (1) HAProxy1 est le nœud privilégié, (2) la ressource à monter est la VIP 192.168.0.10 sur l'interface ens33, et (3) le service haproxy doit être démarré au même moment. Ainsi, HAProxy ne tourne que sur le nœud actif.

```
HAProxy1 ipaddr::192.168.0.10/24/ens33 haproxy
```

- **HAProxy1** : nœud primaire qui prend en charge les ressources au démarrage.
- `ipaddr::192.168.0.10/24/ens33` : ressource IP virtuelle. Heartbeat assigne l'adresse 192.168.0.10/24 sur l'interface ens33. C'est l'adresse que le client CLI-01 utilise.
- **haproxy** : Heartbeat démarre et arrête le service HAProxy en même temps que la VIP. Cela garantit que HAProxy est actif uniquement sur le nœud qui porte la VIP.

Ce fichier doit être strictement identique sur HAProxy1 et HAProxy2.

6.4 Fichier authkeys — Authentification entre nœuds

Ce fichier contient la clé partagée qui permet aux deux nœuds de s'authentifier mutuellement. Il doit avoir les droits 600 (`chmod 600 /etc/ha.d/authkeys`).



```
GNU nano 8.4 /etc/ha.d/authkeys
auth 1
1 md5 greta
```

Figure 18 — Fichier `/etc/ha.d/authkeys`

Ce que je fais : Je crée `/etc/ha.d/authkeys` avec deux lignes (`auth 1` et `1 md5 greta`), puis j'applique `chmod 600 /etc/ha.d/authkeys`.

Pourquoi : Sans authentification, n'importe quelle machine du WAN pourrait se faire passer pour un nœud et provoquer un faux failover. La clé MD5 partagée empêche cela. Les permissions 600 (lisible uniquement par root) sont exigées par Heartbeat, qui refuse de démarrer sinon.

Tests de bascule et validation

Cette étape démontre que l'architecture fonctionne réellement. Tous les tests sont effectués depuis le client CLI-01 (192.168.0.100) en utilisant uniquement l'adresse VIP 192.168.0.10.

7.1 Test 1 — Round-robin entre SERV-WEB1 et SERV-WEB2

On accède à `http://192.168.0.10` depuis le navigateur du client CLI-01 et on rafraîchit plusieurs fois la page. HAProxy doit alterner les réponses entre les deux backends.

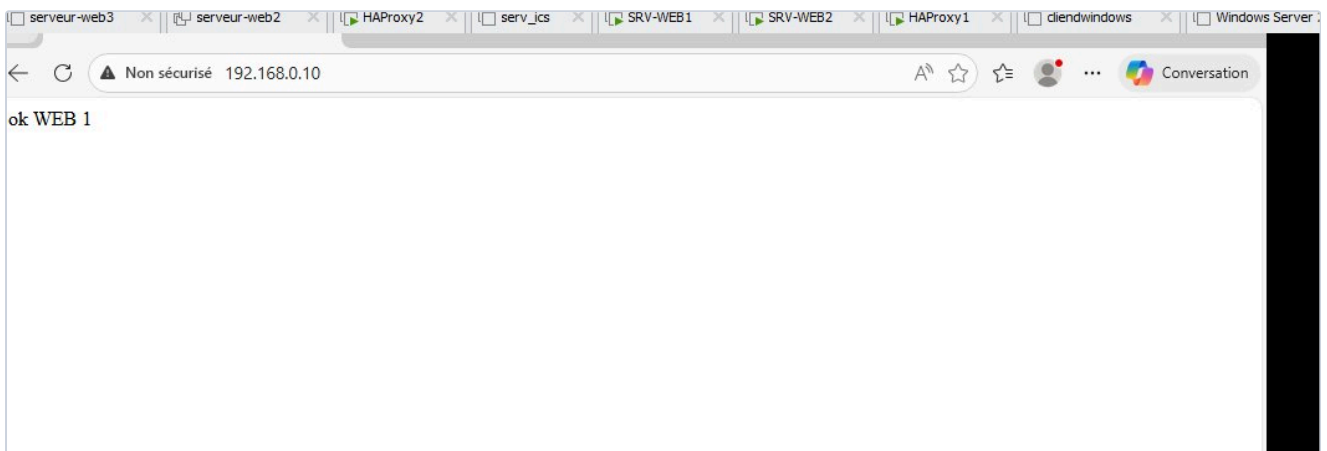


Figure 19 — Accès à la VIP depuis CLI-01 : réponse d'un premier serveur web

Ce que je fais : Depuis le navigateur de CLI-01, je tape l'URL `http://192.168.0.10` et je vois apparaître la page "ok WEB 1".

Pourquoi : Cela prouve deux choses simultanément : (1) la VIP est bien active et joignable depuis le client, et (2) HAProxy a bien relayé la requête vers SERV-WEB1 via le LAN.

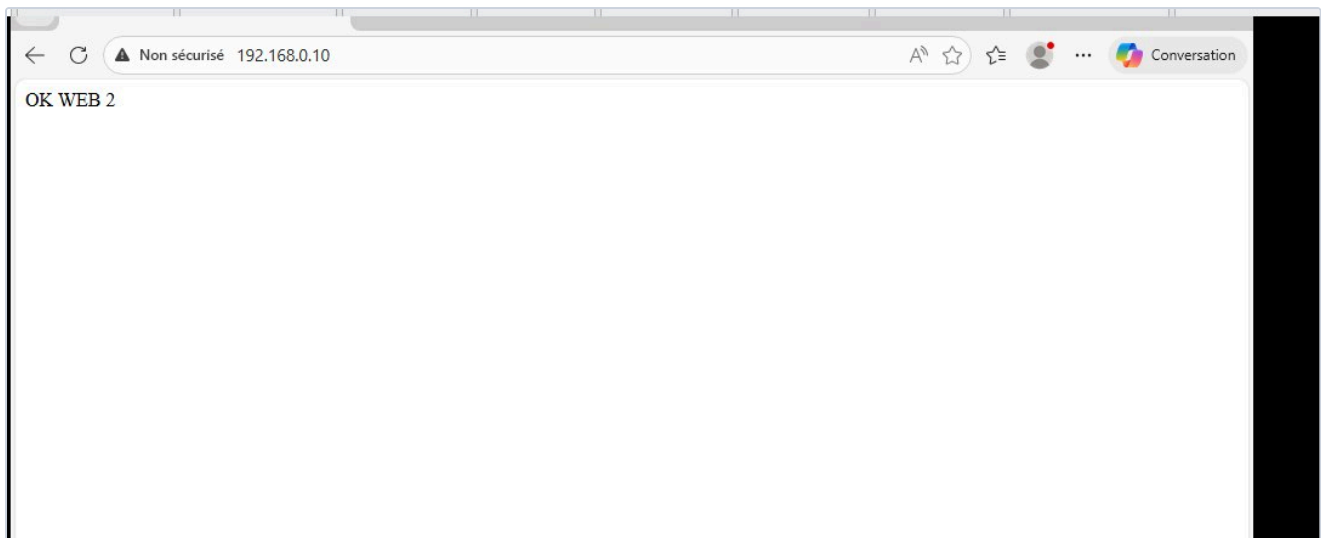


Figure 20 — Après rafraîchissement : réponse de l'autre serveur web

Ce que je fais : Je fais F5 dans le navigateur et la page change : cette fois c'est "OK WEB 2" qui s'affiche.

Pourquoi : C'est la démonstration visuelle du round-robin : la requête suivante a bien été envoyée à l'autre backend. Si j'actualise encore, je reviens sur WEB 1, puis WEB 2, etc. La charge est équitablement répartie.

7.2 Page de statistiques HAProxy

On vérifie également l'état des backends via la page de statistiques HAProxy, accessible sur <http://192.168.0.10:8080/stats> (ou 8404 selon la configuration).

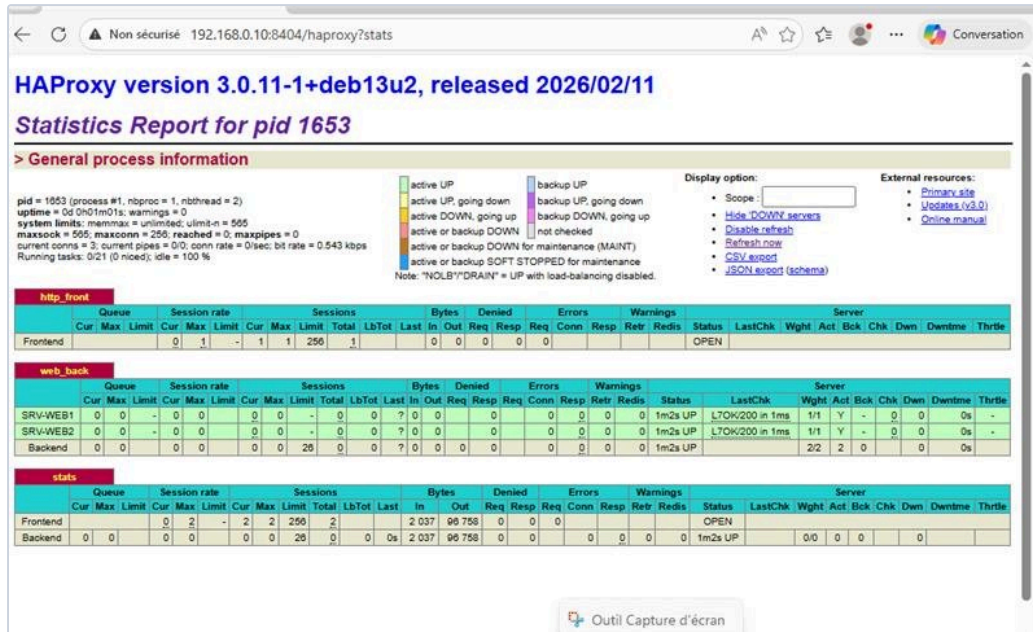


Figure 21 — Page de statistiques HAProxy

Ce que je fais : J'ouvre <http://192.168.0.10:8404/haproxy?stats> et je vérifie que les deux lignes SRV-WEB1 et SRV-WEB2 sont au vert (status UP) avec un LastChk L7OK en quelques millisecondes.

Pourquoi : Cette page fournit une vue temps réel de l'état du cluster : quels backends sont vivants, combien de sessions sont actives, les octets transférés, etc. C'est l'outil d'exploitation de référence pour un administrateur HAProxy.

7.3 Test 2 — Panne de HAProxy1 : bascule de la VIP

On simule une panne du nœud primaire en arrêtant le service Heartbeat sur HAProxy1. Normalement, la VIP doit migrer vers HAProxy2 en quelques secondes et le service doit rester accessible.

```
sudo systemctl stop heartbeat
```

```
root@HAProxy2:~# systemctl stop heartbeat
root@HAProxy2:~#
```

Figure 22 — Arrêt de Heartbeat sur HAProxy1

Ce que je fais : Je me connecte en SSH sur HAProxy1 et je lance `systemctl stop heartbeat` pour simuler la défaillance du nœud primaire.

Pourquoi : En stoppant uniquement Heartbeat (et non toute la VM), on reproduit une vraie panne réseau qui coupe la communication entre les nœuds sans éteindre la machine. C'est le scénario le plus réaliste pour tester le failover.

Après environ 10 secondes (deadtime configuré dans ha.cf), HAProxy2 détecte la disparition de HAProxy1 et prend en charge la VIP. On vérifie sur HAProxy2 avec ip a :

```
root@HAProxy1:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c9:83:e0 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29c983e0
    inet 192.168.0.11/24 brd 192.168.0.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.0.10/24 brd 192.168.0.255 scope global secondary ens33:0
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe09:83e0/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c9:83:ea brd ff:ff:ff:ff:ff:ff
    altname enp2s5
    altname enx000c29c983ea
    inet 172.20.0.11/24 brd 172.20.0.255 scope global ens37
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe09:83ea/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
root@HAProxy1:~#
```

cliquez dans la fenêtre ou appuyez sur Ctrl+G.

Figure 23 — Après la bascule, la VIP est portée par HAProxy2

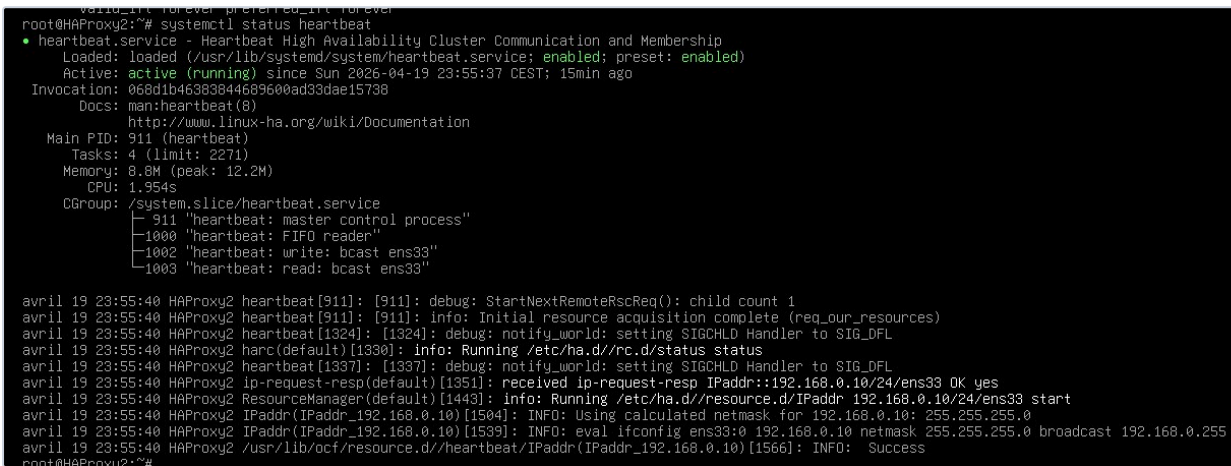
Ce que je fais : Je fais ip a sur HAProxy2 et je constate que l'interface ens33 porte désormais deux adresses : sa propre IP 192.168.0.12/24 ET la VIP 192.168.0.10/24 en scope global secondary.

Pourquoi : C'est la preuve du failover réussi : le nœud secondaire a pris le relais. Pendant ce temps, le client CLI-01 a continué d'accéder à http://192.168.0.10 sans savoir qu'il y avait eu une bascule : la continuité de service est assurée.

7.4 Vérification du statut Heartbeat sur le nœud actif

On peut aussi consulter l'état du service Heartbeat et les logs pour confirmer la bascule :

```
systemctl status heartbeat
journalctl -u heartbeat -n 20
```



```
root@HAProxy2:~# systemctl status heartbeat
• heartbeat.service - Heartbeat High Availability Cluster Communication and Membership
   Loaded: loaded (/usr/lib/systemd/system/heartbeat.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-04-19 23:55:37 CEST; 15min ago
   Invocation: 068d1b46383844689600ad33dae15738
   Docs: man:heartbeat(8)
        http://www.linux-ha.org/wiki/Documentation
   Main PID: 911 (heartbeat)
     Tasks: 4 (limit: 2271)
    Memory: 8.8M (peak: 12.2M)
       CPU: 1.954s
   CGroup: /system.slice/heartbeat.service
           └─ 911 "heartbeat: master control process"
              └─ 1000 "heartbeat: FIFO reader"
                 └─ 1002 "heartbeat: write: bcast ens33"
                    └─ 1003 "heartbeat: read: bcast ens33"

avril 19 23:55:40 HAProxy2 heartbeat[911]: [911]: debug: StartNextRemoteRscReq(): child count 1
avril 19 23:55:40 HAProxy2 heartbeat[911]: [911]: info: Initial resource acquisition complete (req_our_resources)
avril 19 23:55:40 HAProxy2 heartbeat[1324]: [1324]: debug: notify_world: setting SIGCHLD Handler to SIG_DFL
avril 19 23:55:40 HAProxy2 harc(default)[1330]: info: Running /etc/ha.d/rc.d/status status
avril 19 23:55:40 HAProxy2 heartbeat[1337]: [1337]: debug: notify_world: setting SIGCHLD Handler to SIG_DFL
avril 19 23:55:40 HAProxy2 ip-request-resp(default)[1351]: received ip-request-resp IPaddr::192.168.0.10/24/ens33 OK yes
avril 19 23:55:40 HAProxy2 ResourceManager(default)[1443]: info: Running /etc/ha.d/resource.d/IPaddr 192.168.0.10/24/ens33 start
avril 19 23:55:40 HAProxy2 IPaddr(IPaddr_192.168.0.10)[1504]: INFO: Using calculated netmask for 192.168.0.10: 255.255.255.0
avril 19 23:55:40 HAProxy2 IPaddr(IPaddr_192.168.0.10)[1539]: INFO: eval ifconfig ens33:0 192.168.0.10 netmask 255.255.255.0 broadcast 192.168.0.255
avril 19 23:55:40 HAProxy2 /usr/lib/ocf/resource.d/heartbeat/IPaddr(IPaddr_192.168.0.10)[1566]: INFO: Success
root@HAProxy2:~#
```

Figure 24 — Statut et journal de Heartbeat après la bascule

Ce que je fais : Je consulte `systemctl status heartbeat` sur HAProxy2 et je lis dans les logs les lignes "Running /etc/ha.d/resource.d/IPaddr 192.168.0.10/24/ens33 start" et "Success" : Heartbeat a bien monté la VIP.

Pourquoi : Les logs sont la preuve formelle que la bascule a bien eu lieu (et non un simple hasard). En exploitation réelle, on surveille en permanence `/var/log/heartbeat.log` pour détecter toute anomalie de cluster.

7.5 Test 3 — Panne d'un serveur backend

On simule la panne d'un serveur web en arrêtant Apache2 sur SERV-WEB1. HAProxy doit détecter la panne via son health-check (option `httpchk GET /`) et rediriger automatiquement tout le trafic vers SERV-WEB2.

```
sudo systemctl stop apache2 # sur SERV-WEB1
```

À partir de ce moment, toutes les requêtes vers `http://192.168.0.10` renvoient uniquement la page "OK WEB 2". Sur la page de stats, SRV-WEB1 passe en rouge (DOWN), SRV-WEB2 reste vert.

Ce que je fais : J'arrête Apache sur SERV-WEB1, je rafraîchis plusieurs fois `http://192.168.0.10` depuis le client, et je constate que seule la page WEB 2 s'affiche.

Pourquoi : C'est le second niveau de haute disponibilité : même si tous les HAProxy fonctionnent, la panne d'un backend est gérée automatiquement grâce aux health-checks. L'utilisateur ne voit aucune erreur HTTP, juste un léger déséquilibre de charge.

7.6 Test 4 — Retour à la normale (auto_failback)

On redémarre le service Heartbeat sur HAProxy1 :

```
sudo systemctl start heartbeat
```

Grâce à la directive `auto_failback` on du fichier `ha.cf`, la VIP revient automatiquement sur HAProxy1 après quelques secondes. On peut le vérifier avec `ip a` : la VIP 192.168.0.10 réapparaît sur HAProxy1 et disparaît de HAProxy2.

Ce que je fais : Je relance heartbeat sur HAProxy1 et j'observe avec `ip a` que la VIP revient bien chez lui, tandis que HAProxy2 la libère.

Pourquoi : C'est le scénario idéal : après résolution de la panne, le cluster retrouve son état nominal sans intervention humaine. Cela réduit la charge d'exploitation et garantit que le nœud primaire, généralement mieux dimensionné, reprend son rôle.

Bilan et retour d'expérience

8.1 Ce qui a été mis en place

- 5 machines virtuelles configurées sur VMware Workstation avec deux réseaux distincts WAN et LAN.
- Adressage IP statique configuré sur chaque VM via `/etc/network/interfaces`.
- HAProxy installé et configuré en load balancer round-robin vers deux serveurs web.
- Page de statistiques HAProxy accessible sur le port 8080.
- Heartbeat configuré en cluster actif/passif avec VIP 192.168.0.10 flottante.
- **Failover automatique** : la VIP bascule de HAProxy1 vers HAProxy2 en cas de panne.
- Retour automatique du nœud primaire validé (`auto_failback on`).

8.2 Limites de l'architecture

- **Split-brain** : si la communication entre HAProxy1 et HAProxy2 est coupée, les deux nœuds peuvent croire que l'autre est mort et revendiquer la VIP simultanément.
- Heartbeat est un logiciel ancien, moins activement maintenu que ses successeurs.
- Les serveurs web (SERV-WEB1 et SERV-WEB2) n'ont pas de haute disponibilité entre eux pour les données (pas de synchronisation de fichiers).
- L'authentification Heartbeat utilise MD5, un algorithme obsolète aujourd'hui (SHA1 ou SHA256 seraient préférables).